

Guía de Validaciones y Sanitización en PHP

Índice

1. Introducción
2. Validación de Datos
 - 2.1. filter_var()
 - 2.2. preg_match()
 - 2.3. is_int()
 - 2.4. is_numeric()
3. Sanitización de Datos
 - 3.1. filter_var()
 - 3.2. htmlspecialchars()
 - 3.3. strip_tags()
 - 3.4. trim()
 - 3.5. addslashes()
4. Protección Contra Inyecciones SQL
 - 4.1. Sentencias Preparadas con PDO
 - 4.2. Sentencias Preparadas con MySQLi
5. Protección Contra CSRF (Cross-Site Request Forgery)
 - 5.1. Generar un Token CSRF
6. Protección Contra XSS (Cross-Site Scripting)
 - 6.1. htmlspecialchars()
 - 6.2. strip_tags()
7. Verificación de Archivos Subidos
 - 7.1. is_uploaded_file()
 - 7.2. mime_content_type()

8. Validación de Fechas

8.1. DateTime::createFromFormat()

9. Resumen

1. Introducción

En el desarrollo web moderno, la validación y sanitización de los datos proporcionados por los usuarios es una de las tareas más críticas

para proteger las aplicaciones contra vulnerabilidades comunes. Este documento presenta las mejores prácticas y funciones en PHP

para realizar estas tareas de manera eficiente, asegurando la integridad de los datos y la seguridad de la aplicación.

2. Validación de Datos

La validación de datos asegura que los valores recibidos sean correctos y que cumplan con los criterios especificados.

A continuación, se presentan varias funciones modernas de PHP para validar distintos tipos de datos.

2.1. **filter_var()**

La función `filter_var()` es la más versátil para validar y sanitizar datos. Permite utilizar filtros predefinidos para garantizar que los datos sean correctos según el tipo.

```
```php
// Validar un correo electrónico

$email = filter_var($email, FILTER_VALIDATE_EMAIL);

// Validar un número entero

$int = filter_var($integer, FILTER_VALIDATE_INT);

// Validar una URL

$url = filter_var($url, FILTER_VALIDATE_URL);

// Validar una dirección IP

$ip = filter_var($ip, FILTER_VALIDATE_IP);
```
```

2.2. **preg_match()**

Utiliza expresiones regulares para realizar validaciones más complejas.

```
```php
// Validar una fecha en formato YYYY-MM-DD
if (preg_match("/^\d{4}-\d{2}-\d{2}$/", $fecha)) {
 // La fecha es válida
}
```
```

2.3. **is_int()**

Verifica si una variable es un número entero.

```
```php
if (is_int($number)) {
 // Es un entero
}
```
```

2.4. **is_numeric()**

Verifica si una variable es numérica.

```
```php
if (is_numeric($data)) {
 // Es numérico
}
```
```

3. Sanitización de Datos

La sanitización de datos asegura que los valores sean limpiados de caracteres no deseados o peligrosos.

Aquí se muestran algunas funciones de PHP para sanitizar datos de entrada.

3.1. **filter_var()**

```
```php
```

```
// Sanitizar correo electrónico
```

```
$email = filter_var($email, FILTER_SANITIZE_EMAIL);
```

```
// Sanitizar una URL
```

```
$url = filter_var($url, FILTER_SANITIZE_URL);
```

```
// Sanitizar un número entero
```

```
$int = filter_var($int, FILTER_SANITIZE_NUMBER_INT);
```

```
// Sanitizar texto
```

```
$text = filter_var($text, FILTER_SANITIZE_STRING);
```

```
```
```

3.2. **htmlspecialchars()**

Convierte caracteres especiales en entidades HTML para evitar ataques XSS.

```
```php
```

```
$safeString = htmlspecialchars($string, ENT_QUOTES, 'UTF-8');
```

```
```
```

3.3. **strip_tags()**

Elimina todas las etiquetas HTML y PHP de una cadena.

```
```php
$cleanString = strip_tags($string);
```
```

3.4. **trim()**

Elimina los espacios en blanco al principio y al final de una cadena.

```
```php
$cleanString = trim($string);
```
```

3.5. **addslashes()**

Escapa caracteres especiales con barras invertidas.

```
```php
$escapedString = addslashes($string);
```
```

4. Protección Contra Inyecciones SQL

4.1. **Sentencias Preparadas con PDO**

El uso de sentencias preparadas es la forma más segura de interactuar con bases de datos.

```
```php
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
$stmt->bindParam(':email', $email, PDO::PARAM_STR);
$stmt->execute();
```
```

4.2. **Sentencias Preparadas con MySQLi**

```
```php
$stmt = $conn->prepare("SELECT * FROM users WHERE email = ?");
$stmt->bind_param("s", $email);
$stmt->execute();
```
```

5. Protección Contra CSRF (Cross-Site Request Forgery)

5.1. **Generar un Token CSRF**

Genera un token único que se incluye en los formularios para evitar ataques CSRF.

```
```php
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```
```

6. Protección Contra XSS (Cross-Site Scripting)

6.1. **htmlspecialchars()**

```
```php
```

```
$safeData = htmlspecialchars($data, ENT_QUOTES, 'UTF-8');
```

```
```
```

6.2. **strip_tags()**

```
```php
```

```
$cleanData = strip_tags($data);
```

```
```
```

7. Verificación de Archivos Subidos

7.1. `is_uploaded_file()`

Verifica si el archivo fue subido correctamente.

```
```php
if (is_uploaded_file($file['tmp_name'])) {
 // El archivo fue subido
}
```
```

7.2. `mime_content_type()`

Verifica el tipo MIME del archivo para asegurarse de que es el esperado.

```
```php
$mimeType = mime_content_type($file['tmp_name']);
if ($mimeType === 'image/jpeg') {
 // Es una imagen JPEG
}
```
```

8. Validación de Fechas

8.1. **DateTime::createFromFormat()**

Utiliza esta función para validar fechas con un formato específico.

```
```php

$date = DateTime::createFromFormat('Y-m-d', $dateString);

if ($date && $date->format('Y-m-d') === $dateString) {

 // La fecha es válida

}

```
```

9. Resumen

Las funciones PHP modernas proporcionadas en este documento son esenciales para asegurar que los datos recibidos a través de formularios sean válidos y estén sanitizados, reduciendo riesgos como inyecciones SQL y ataques XSS. Utiliza siempre estas funciones y mejores prácticas para mantener la seguridad de tus aplicaciones web.